TRAINING FEEDBACK

Day 1 | Monday, 26 October 2026

Course introduction and first C++ program

The training began with an introduction to the NC(V) Level 3 Technical Programming course. We looked at the work covered during the week and how to teach programming in small steps. This helped me see the importance of starting with a clear plan before asking learners to write code.

We were introduced to the C++ programming environment and the steps used to write, build and run a program in Code::Blocks. We also discussed Git and GitHub as tools for keeping track of changes and saving practical work. I learned that saving each working version makes it easier to return to earlier code when errors occur.

The first coding activities helped me become more familiar with the structure of a C++ program and how to read the output. I was reminded that learners need time to practise simple programs before moving to longer tasks.

Day 2 | Tuesday, 27 October 2026

Working with data, calculations and input

On Tuesday, we focused on data types, variables, constants and calculations in C++. We compared whole numbers and decimal numbers and saw how the choice of data type affects the answer. We also practised the order of operations and used formatting tools to display results clearly.

We learned how to accept input using cin and how to handle incorrect values. The maths library was used for functions such as pow(), sqrt() and round(). In the afternoon, we planned a rainwater-tank program using an IPO chart, a flowchart and test values before writing the code.

The practical task involved calculating the tank volume, capacity in litres and surface area. We tested the program using normal, zero, negative and incorrect inputs. This taught me the value of checking input and testing each part of a program as it is built.

Day 3 | Wednesday, 28 October 2026

Selection, loops and a statistics program

Wednesday focused on making decisions and repeating steps in C++. We worked with selection and repetition structures and discussed when to use if statements, switch, while, for and do-while loops. The examples showed how a program can make choices based on the values entered by a user.

In the afternoon, we built a menu-driven program for capturing learner marks. The program had options to enter a mark, show statistics and exit. We used a validation loop to accept marks from 0 to 100 and calculated the count, average, highest, lowest and number of competent learners.

We also looked at common errors, such as using the wrong starting value for the highest or lowest mark and dividing by zero when no marks have been entered. I found the debugging tasks useful because they showed how to trace a program carefully and test it with different values.

Day 4 | Thursday, 29 October 2026

Functions, arrays and strings

On Thursday, we learned how to divide a large program into smaller functions. We worked with function prototypes, definitions, calls, parameters and return values. The examples helped me understand the difference between a void function and a function that returns a value.

We practised writing functions for calculations and explored how values are passed to functions. We also worked with one-dimensional arrays and strings, including reading and processing several values. These activities showed how functions can make code easier to read, test and reuse.

One important lesson for me was that each function should have a clear task. Instead of placing all the code inside main(), we can use smaller functions and call them when needed. This is an approach I can use when teaching learners to build longer programs.

Day 5 | Friday, 30 October 2026

Classes, objects and an integrated project

On the last day, we were introduced to object-oriented programming in C++. We learned about classes, objects, attributes, methods and constructors. We also discussed private and public members, getters and setters, and why a setter should check values before storing them.

We used a spaza shop as an example and planned a small till program. In the practical session, the Product class included a product code, description and price. We used objects in an array, searched for a product by code and calculated the cost of items purchased. Input validation and testing were also part of the task.

The week ended with project planning, GitHub work and checking the practical assessment evidence. I gained useful examples of how basic C++ topics can be combined in one program. I can use these ideas to guide learners from simple tasks to a complete project.